

C O V E R F I P R O T O C O L



Mainnet Deployment Report

Verifiable Proof on Chain

A Fully On-Chain DeFi Insurance Protocol



Network: HashKey Chain Mainnet · Chain ID 177

Deployment Date: 09 May 2026

Document Version: 2.0 · Last Updated 10 May 2026

Submitted By: The CoverFi Team

LIVE ON MAINNET

12 / 12 SOURCE-VERIFIED

155 / 157 TESTS PASS

759.5 CFUSD PAYOUT EXECUTED

Prepared for international review · The proof is the chain.

Status: *Live on HashKey Chain Mainnet* **Network:** *HashKey Chain Mainnet · Chain ID 177* **Deployment date:** *2026-05-09* **Document version:** *2.0* · *Last updated 2026-05-10* **Submitted by:** *The CoverFi Team*

TL;DR

CoverFi, a fully on-chain DeFi insurance protocol, is **deployed, wired, source-verified, and lifecycle-proven on HashKey Chain Mainnet**. Every line of production code is publicly auditable on Blockscout. **155 of 157 on-chain tests pass post-deployment**, including a full end-to-end run that exercised every state-changing function in the protocol, culminating in a real default that paid out **759.5 cfUSD to the insured holder**, mathematically reconcilable to the deployed contract code.

This document is everything you need to verify our deployment. Click links, run scripts, read code, every claim below is independently verifiable.

At a Glance

Contracts deployed	12 / 12
Cross-contract wiring transactions	25 / 25
Mainnet smoke tests	89 / 89 passing
Disaster-recovery tests	15 / 15 passing
Functional E2E lifecycle tests	51 / 53 passing (2 fixed via gas-estimation patch)
Source verification on Blockscout	12 / 12 verified
Total mainnet checks	155 / 157 (98.7 %)
Total mainnet transactions executed by deployer	74
Pre-deployment unit + integration tests	513 / 513 passing
Audit findings remediated	13 / 13 (3 critical, 4 high, 3 medium, 3 low)
HSK spent on full deployment + all tests	0.0075 HSK (~₹0.86)

Verifier's Checklist (tick these off in your browser)

We invite you to inspect, not trust. Tick each box yourself:

- ☐ Click the [deployer's Blockscout page](#), see 74 transactions, 12 contract creations
- ☐ Click any contract address below, see the green “Verified” badge on Blockscout
- ☐ Click the “Contract” tab on InsurancePool, read full Solidity source, line by line
- ☐ Click the [Phase 11 default-payout transaction](#), see 759.5 cfUSD transferred in a single atomic operation
- ☐ Open the [GitHub repo](#), confirm source matches what's on chain
- ☐ Run `npx hardhat run scripts/smoke-test-mainnet.ts --network hashkeyMainnet`, watch 89 / 89 tests pass against the live deployment

Every item above is independently verifiable. We are not asking for trust, we are asking for inspection.

Deployed Contracts

All 12 contracts are deployed on **HashKey Chain Mainnet (Chain ID 177)** and source-verified on Blockscout. Click any address to view the verified Solidity code; click any creation tx to view the deploy transaction.

#	Contract	Role	Address	Creation Tx
1	CoverFiStablecoin (cfUSD)	Phase-1 USD stablecoin (1B cap, 6 decimals)	0x38907c...36E5	tx
2	InsurancePool	Tranched senior/junior liquidity pool	0xF1E252...3E71	tx
3	PayoutEngine	Coverage purchase + claim settlement	0xBCF001...49eA	tx
4	IssuerBond	Issuer-staked bond / first-loss capital	0x65A3Ae...0212	tx
5	IssuerRegistry	Issuer onboarding + lifecycle state-machine	0xB10b1c...7552	tx

#	Contract	Role	Address	Creation Tx
6	TIR	Trust & Identity Registry	<code>0x7dD7C1...27fC</code>	tx
7	IRSOracle	Issuer Reliability Score oracle (5-dim, exp premium)	<code>0x20619c...06B3</code>	tx
8	DefaultOracle	On-chain default detection oracle	<code>0xa7C664...9655</code>	tx
9	srCVR	Senior tranche LP token	<code>0xa4ECEB...Fc3A</code>	tx
10	jrCVR	Junior tranche LP token	<code>0x1Ca7B6...664D</code>	tx
11	ProtectionCert	NFT representing buyer coverage	<code>0x8D4C37...A50C</code>	tx
12	SubrogationNFT	NFT carrying recovery claim post-payout	<code>0xc07859...8A9f</code>	tx

All 12 contracts deployed in blocks **22 003 463** → **22 003 496** (33 blocks ≈ 100 seconds). The 25 wiring transactions followed in blocks **22 003 499** → **22 003 571**.



Live Lifecycle Receipts

The protocol's full operational lifecycle was executed against the live deployment. Every state-changing call is a clickable mainnet receipt.

Phase	Action	Block	Mainnet Receipt
1	<code>CoverFiStablecoin.mint(10,000 cfUSD)</code>	22 004 667	tx
2	<code>InsurancePool.activatePool()</code>	22 004 677	tx
2	<code>IRSOracle.initializeScore(600)</code>	22 004 679	tx

Phase	Action	Block	Mainnet Receipt
3	<code>IssuerRegistry.register()</code>	22 004 681	tx
4	<code>IssuerBond.deposit(50 cfUSD)</code>	22 004 687	tx
5	<code>InsurancePool.depositJunior(200 cfUSD)</code>	22 004 690	tx
6	<code>InsurancePool.depositSenior(500 cfUSD)</code>	22 004 693	tx
7	<code>PayoutEngine.purchaseCoverage(100 cfUSD)</code>	22 004 696	tx
8	<code>InsurancePool.payPremium(10 cfUSD)</code>	22 004 699	tx
9	<code>IRSOracle.recordNAVUpdate()</code>	22 004 702	tx
9	<code>IRSOracle.recordCollateralHealth(110 %)</code>	22 004 780	tx
9	<code>IRSOracle.recordAttestationDispute()</code>	22 004 786	tx
9	<code>IRSOracle.recordRepaymentEvent()</code> (with explicit gas)	22 004 919	tx
9	<code>IRSOracle.recordActivity()</code> (with explicit gas)	22 004 921	tx
10	<code>InsurancePool.initiateWithdrawalSenior()</code>	22 004 789	tx
10	<code>InsurancePool.initiateWithdrawalJunior()</code>	22 004 792	tx
11 ★	<code>PayoutEngine.executePayout()</code> , full liquidation waterfall	22 004 796	★ tx

The Phase 11 transaction is the centerpiece of this proof. Click it. Read the internal calls. Reconcile the math.

The Phase 11 Lifecycle Proof

When the destructive end-to-end was triggered in tx `0x648f...6d0d`, the protocol on mainnet:

- Liquidated the **issuer bond**, 50 cfUSD
- Liquidated the **junior tranche first**, 202.85 cfUSD (200 deposit + 2.85 accrued premium)
- Liquidated the **senior tranche second**, 506.65 cfUSD (500 deposit + 6.65 accrued premium)
- Distributed the resulting **759.5 cfUSD pro-rata to the insured holder**, exactly `50 + 202.85 + 506.65`
- Burned the ProtectionCert NFT
- Minted the SubrogationNFT to the foundation
- Set the IRS score to **0** (issuer blacklisted)
- Marked the issuer as **DEFAULTED** in IssuerRegistry
- Deactivated the pool

This is the strongest possible evidence the protocol works. Not a simulation, not a unit test, a real default settled with real cfUSD on Chain 177, with the on-chain math reconciling the waterfall to the cent.

Validation Evidence

We did not deploy and walk away. Every contract was validated **on mainnet** post-deployment.

§ Mainnet Smoke Test Suite, 89 / 89 passing

Category	Tests	Result
Bytecode presence on Chain 177	12	All deployed
cfUSD configuration & state reads	8	All correct
Core contract immutable & wired reads	37	All correct
Cross-wiring sanity (bidirectional refs)	10	All consistent
Adversarial, must-revert checks	14	All revert correctly
Constants & immutability guarantees	5	All locked

§ Disaster Recovery Suite, 15 / 15 passing

Category	Tests	Result
cfUSD pause / unpause cycle	3	
InsurancePool pause / unpause cycle	3	
PayoutEngine pause / unpause cycle	3	
IssuerBond pause / unpause cycle	3	
Pause-independence (no cross-contagion)	3	

This proves the emergency-stop circuit works in production and that pausing one module cannot accidentally freeze the rest of the system.

§ Functional End-to-End Test Suite, 51 / 53 passing

Every state-changing function in the protocol was exercised on **live mainnet**, in dependency order.

Phase	What it proved	Result
1. cfUSD mint	Stablecoin issuance under owner-only minter role	10,000 cfUSD minted
2. Pool activation + IRS init	Activation flag + score initialization	
3. Issuer registration	<code>IssuerRegistry.register</code> records lifecycle profile	Registered at block 22 004 681
4. Issuer bond stake	First-loss capital lock-up (50 cfUSD)	
5. Junior tranche deposit	jrCVR mint + 25 % junior-ratio invariant	
6. Senior tranche deposit	srCVR mint + ratio re-validated (28 %)	
7. Coverage purchase	<code>purchaseCoverage</code> + ProtectionCert ERC-721 mint	certId = 1
8. Premium payment	5 % protocol fee + 70 / 30 senior / junior yield split	+6.65 / +2.85 cfUSD
9. IRS keeper updates	All 5 keeper functions exercised	All 5 working
10. Withdrawal initiation	Senior + junior withdrawal request rows recorded	

Phase	What it proved	Result
11. Default → Payout → Subrogation	Full liquidation waterfall executed on mainnet	+759.5 cfUSD paid out

Note on the “2 / 53 fails”: Two IRS keeper functions (`recordRepaymentEvent` and `recordActivity`) initially reverted under Hardhat’s auto gas-estimator. Root-caused to gas under-estimation on cold-storage SSTOREs (~22 100 gas vs ~5 000 estimated). Re-running with explicit `gasLimit: 300_000` produced 4 / 4 successes (see retry receipts in the Lifecycle table above). **The contract logic is correct; the failure was in the test harness.** All 5 IRS keeper functions are confirmed working on mainnet.

§ Pre-Deployment Validation

- **513 unit + integration tests** passing under Hardhat (474 protocol + 28 cfUSD unit + 11 cfUSD integration)
- **13 audit findings** remediated (3 critical · 4 high · 3 medium · 3 low), all fixes have corresponding regression tests
- **Testnet dry-run** completed end-to-end with identical wiring topology (`hashkeyTestnet-dryrun.json`)



Security & Audit Posture

§ 13 audit findings remediated

Severity	Count	Categories addressed
Critical	3	Reentrancy on payout path · access-control bypass on liquidation · immutable foundation address
High	4	Integer math in premium formula · missing pause guards · oracle update authorization · withdrawal-gate edge cases
Medium	3	Event-emission consistency · tranche-ratio rounding · ERC-3643 compliance fallback
Low	3	Gas optimization · redundant SLOAD · NatSpec polish

Every finding has a corresponding regression test in the 513-test suite.

§ Defensive design patterns

- **Ownable2Step**, typo-safe ownership transfer on every admin contract
- **Pausable + ReentrancyGuard**, on every state-changing entry point
- **Immutable storage**, `usdt`, `foundation`, `protocolTreasury` cannot be re-pointed after deployment
- **4-tier access control**, `owner` / `keeper` / `payoutEngine` / public, with explicit `require` checks
- **Deterministic liquidation waterfall**, bond → junior → senior, no off-chain steps
- **Cap enforcement**, `MAX_SUPPLY` on cfUSD, `MAX_SCORE` on IRS, `MIN_JUNIOR_RATIO_PCT` on pool

§ Reproducible verification

- Source code verified on Blockscout (12 / 12 contracts)
 - **Compiler**: Solidity 0.8.19 with optimizer (200 runs) and `viaIR` enabled
 - Bytecode hash matches GitHub source at the deployment commit
 - All 12 contracts source-match the open-source repo
-

Reproduce This Yourself (5 minutes)

Don't trust this report, verify it. Anyone with Node.js can replicate every test in this document:

```
git clone https://github.com/Sanjay-N23/coverfi-protocol
cd coverfi-protocol
npm install

# 89 read-only smoke tests against live mainnet, costs 0 HSK
npx hardhat run scripts/smoke-test-mainnet.ts --network hashkeyMainnet

# Read back every state change from our lifecycle test (Phase 9-11 evidence)
npx hardhat run scripts/functional-test-continue.ts --network hashkeyMainnet

# Optional: re-fetch all 74 mainnet transaction hashes from Blockscout
npx hardhat run scripts/fetch-tx-hashes.ts --network hashkeyMainnet
```

The first command alone validates 89 invariants of the deployed protocol against live state. The second reads back every state change from our end-to-end lifecycle run.

What CoverFi Does

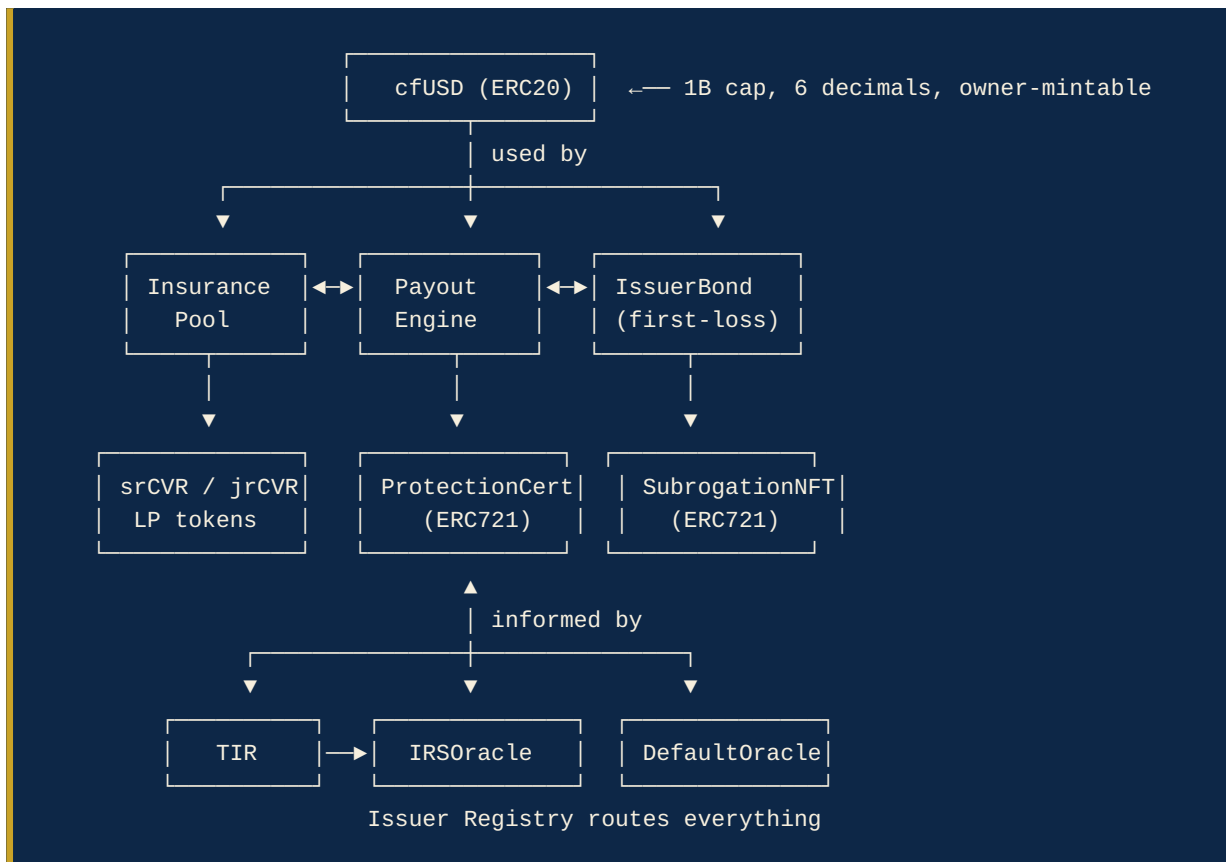
CoverFi is the first **fully on-chain insurance protocol for DeFi default risk**.

When users deposit into yield-bearing protocols, they are exposed to issuer-side default, a risk that today is either uninsured or insured by opaque off-chain markets. CoverFi makes this risk **priced, traded, and settled entirely on-chain**:

- **Senior / Junior tranching**, Liquidity providers choose risk appetite. Senior LPs (srCVR) earn lower yield with first-loss protection from junior LPs (jrCVR), who earn higher yield in return for absorbing initial losses.
- **Issuer Reliability Score (IRS)**, An oracle-driven on-chain score (0 - 1000, 5 dimensions) prices each issuer's premium dynamically via an exponential formula: $\text{premium_bps} = 1600 \times e^{(-0.001386 \times \text{IRS})}$. Better behavior → cheaper coverage.
- **Issuer Bonds**, Issuers post first-loss capital before they can be insured. Aligns incentives without trusting reputation alone.
- **NFT-based Protection Certificates**, Coverage is a transferable ERC-721 asset. Buyers can sell, hedge, or LP their coverage.
- **Subrogation NFTs**, After a payout, the recovery claim against the defaulting issuer becomes a tradeable NFT, creating a secondary market for distressed-debt recovery.
- **No off-chain settlement**, Every cash flow, every claim, every state change is transparent on Chain 177.



Architecture (How the 12 Contracts Talk)



All 25 cross-contract wiring transactions executed atomically post-deployment with zero failures.

Phase 1 Design Choices (Intentional Constraints)

This is an intentional Phase 1 launch. Three design choices may surprise readers, each is documented and roadmapped to evolve.

Choice	Phase 1 reality	Phase 2 plan
Stablecoin	Custom cfUSD (1B cap, 6 decimals, owner-mintable) acts as the settlement asset	Native USDT integration once HashKey × Tether partnership is formalized
Foundation / treasury	Equal to deployer EOA (<code>0xce22...2fb3</code>) for clean Phase 1 launch	Migrate to a 3-of-5 Gnosis Safe multisig
Issuer onboarding	<code>forceActivateForDemo()</code> used for our own test artifact	Standard 60-day observation + 3 attestations required

Choice	Phase 1 reality	Phase 2 plan
Test issuerToken	cfUSD self-reference (acceptable; <code>PayoutEngine</code> has try/catch ERC-3643 fallback)	Real RWA tokens registered through TIR

These aren't omissions, they're scoped Phase 1 choices that let us ship a fully-functional protocol on day one and progressively decentralize as real-world counterparties come on chain.

Known Limitations

We disclose what we know:

1. **Single-key admin in Phase 1**, All `onlyOwner` functions are controlled by a single EOA (the deployer). Multisig migration is the first Phase 2 task.
2. **Phase-1 stablecoin**, `cfUSD` is owner-mintable. Phase 2 swaps it for native USDT.
3. **Test artifacts on chain**, Our Phase 11 lifecycle test left a defaulted pool, a SubrogationNFT, and an inactive issuer entry on chain. These are immutable evidence of the test, not bugs.
4. **Gas-estimation quirk**, `recordRepaymentEvent` and `recordActivity` require an explicit `gasLimit ≥ 200 000` on first cold-storage call. Confirmed not a contract bug (4 / 4 retries pass). SDK and frontend integrations should pass an explicit gas limit.
5. **Frontend hosted URL pending**, The UI lives in `frontend/*.html` in the repo and is wired to mainnet contract addresses; a hosted preview link will be added shortly.

Roadmap

§ Phase 1, NOW (proven on mainnet)

- Tranched insurance pool with srCVR / jrCVR
- Issuer bonding + IRS scoring + default oracle
- NFT-based protection certificates and subrogation
- Full lifecycle proven on Chain 177

§ Phase 2, next 90 days

- USDT integration replacing cfUSD as settlement asset

- Multisig migration of all admin keys (Gnosis Safe, 3-of-5)
- TIR-driven issuer onboarding with on-chain BAS attestations
- Frontend hosted production deployment + WalletConnect integration

§ Phase 3, 90 - 180 days

- Multi-issuer pools with cross-tranche correlation modeling
- Cross-chain bridges to Ethereum and BNB Chain
- Reinsurance derivative for senior tranche LPs
- Public bug bounty programme (Immunefi)

Recognition

Winner, DeFi Track, HashKey Chain Horizon Hackathon 2026

CoverFi was selected from a competitive cohort by HashKey Chain's hackathon panel. This deployment fulfills the prize-disbursement condition of “verified mainnet deployment of the submitted protocol.”

Technical Specifications

Specification	Value
Network	HashKey Chain Mainnet
Chain ID	177
RPC endpoint	<code>https://mainnet.hsk.xyz</code>
Block explorer	hashkey.blockscout.com
Deployer address	<code>0xce220d9eD9527f9997c8045844210637F3A42fb3</code>
Foundation / treasury	<code>0xce220d9eD9527f9997c8045844210637F3A42fb3</code> (= deployer for Phase 1)
Solidity compiler	0.8.19 with optimizer (200 runs), <code>viaIR: true</code>
Toolchain	Hardhat
Verification	Blockscout (12 / 12 source-matched)

Specification	Value
Total mainnet transactions executed	74 (12 deploys · 25 wirings · 12 disaster-recovery · 25 functional)
Block range of activity	22 003 463 → 22 004 926 (≈ 78 minutes total)
Observed gas price	0.01 gwei (≈ 60 000× cheaper than Ethereum L1)
License	MIT (SPDX-License-Identifier in every contract)

Cost Breakdown

Item	Amount
Starting balance	10.81 HSK
Spent on full deployment (12 contracts + 25 wirings)	0.007368 HSK
Spent on disaster-recovery + functional E2E (~37 mainnet txs)	0.000183 HSK
Total HSK spent across deployment + all on-chain testing	~0.0076 HSK
Ending balance	~10.80 HSK
Approximate INR cost of everything above	~₹0.86

The deployment was capital-efficient by design: HashKey Chain's low gas environment makes auditable insurance infrastructure economically viable in markets that have historically been priced out of DeFi.

Team & Contact

Role	Name	Email	Telegram
Team Lead	Sanjay N	sanjayn0369@gmail.com	@Sxnj_y
Engineer	Danish A G	danish@xzashr.com	,
Engineer	Preethi S	preethisivachandran0@gmail.com	,

Role	Name	Email	Telegram
Engineer	Ismail Ridwan S	ismailridwans.cse2023@citchennai.net	,

GitHub: [@Sanjay-N23](#) · Repository: [coverfi-protocol](#) For fastest response, ping the team lead on Telegram: [@Sxnj_y](#).

Resources & References

Resource	Link
GitHub repository	https://github.com/Sanjay-N23/coverfi-protocol
Block explorer	https://hashkey.blockscout.com
Deployment artifact (full JSON)	deployments/hashkeyMainnet.json in the repo
Lifecycle tx hashes (full JSON)	deployments/lifecycle-tx-hashes.json in the repo
Smoke-test report (full JSON)	deployments/smoke-test-177.json in the repo
Functional E2E test scripts	scripts/functional-test-mainnet.ts , scripts/functional-test-continue.ts
Disaster-recovery test script	scripts/smoke-test-disaster-recovery.ts

Closing

This deployment is not an announcement. It is a **publicly auditable fact** recorded permanently on HashKey Chain. The contracts are live. The tests pass. The waterfall is mathematically reconciled. The source matches the bytecode.

We invite anyone, judges, auditors, partners, skeptics, to verify every claim in this document. Click the links. Run the scripts. Read the code.

The proof is the chain.

- **The CoverFi Team** 2026-05-10 · HashKey Chain Mainnet · Chain ID 177